DATA VISUALIZATION FOR DECISION MAKING

Practical Report 12

Student Name: Jeel Patel

Enrollment No.: T26084801021 / 26084801012

Course: M. Sc. IT (Data Science)

Semester: 1

Course Code: P91A3DVD

Academic Year: 2026-27

Practical 12: High-Density Report Using DAX

Objective: Create a high-performance dashboard using DAX measures and optimized visualizations.

Step 1: Set Up Data Model & Measure Table

- Loaded sales dataset (`Sales_Fact_Practical_12.csv`) into Power BI Desktop and reviewed source schema.
- Created a calculated calendar table `daxDim_Date` using `ADDCOLUMNS` and `CALENDAR` covering dates from 2024 through 2026.
- Established a 1-to-Many active relationship between `daxDim_Date[Date]` and `Sales_Fact_Practical_12[Date]` in Model View.

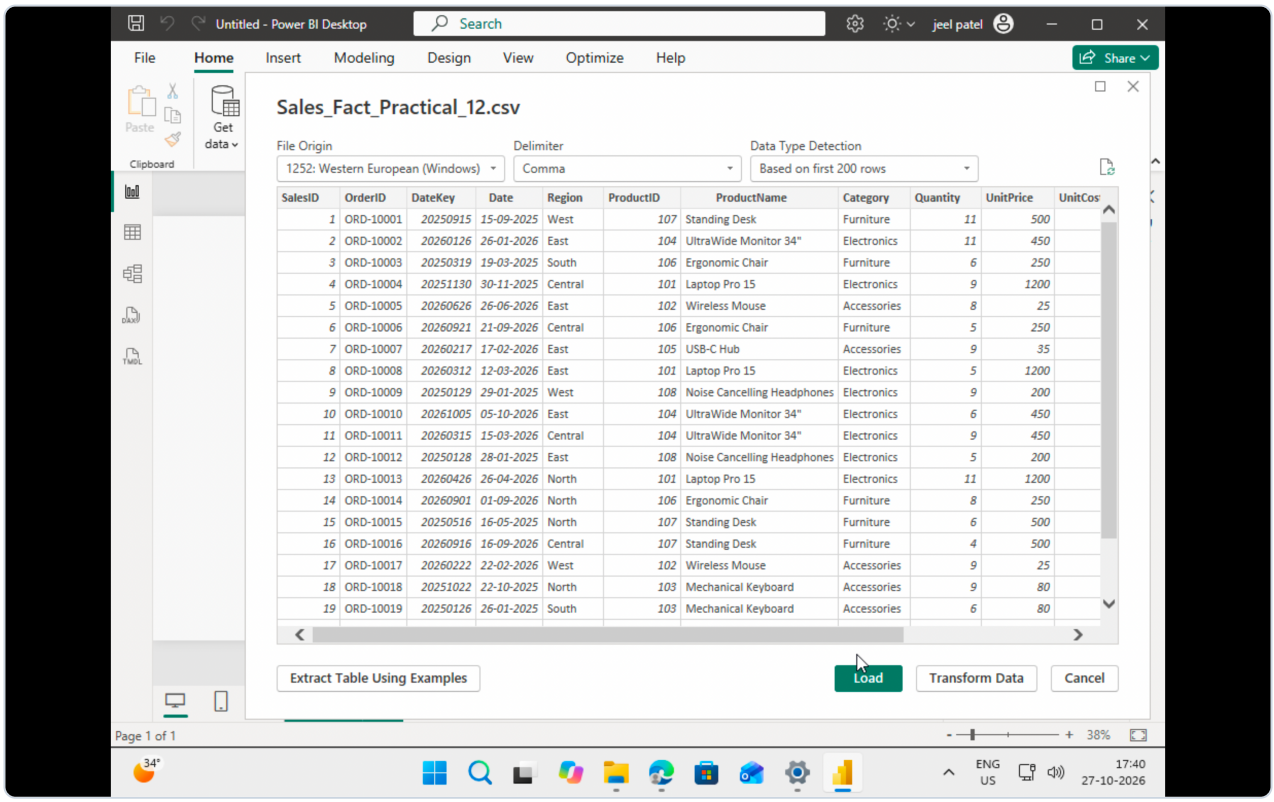


Figure 1.1: Loading Sales_Fact_Practical_12.csv dataset into Power BI Desktop

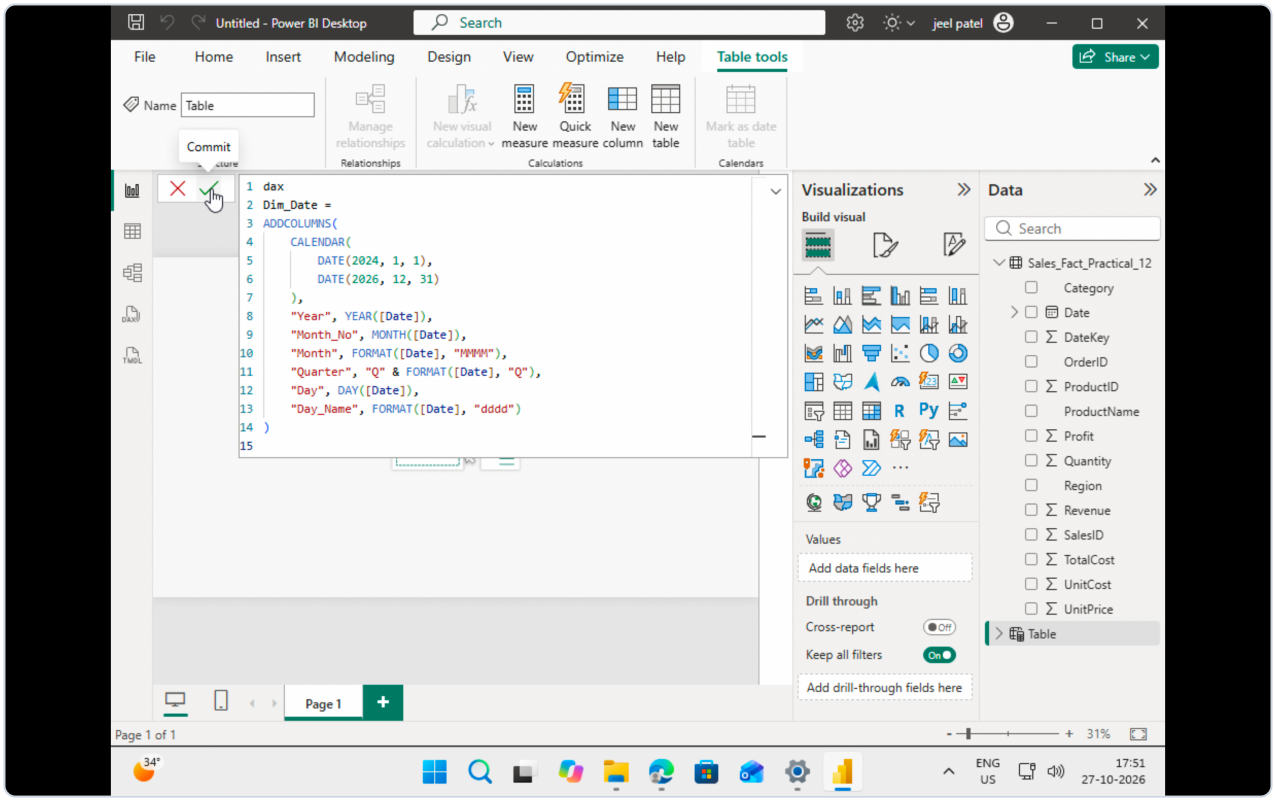


Figure 1.2: Creating daxDim_Date calculated table and defining 1-to-Many relationship

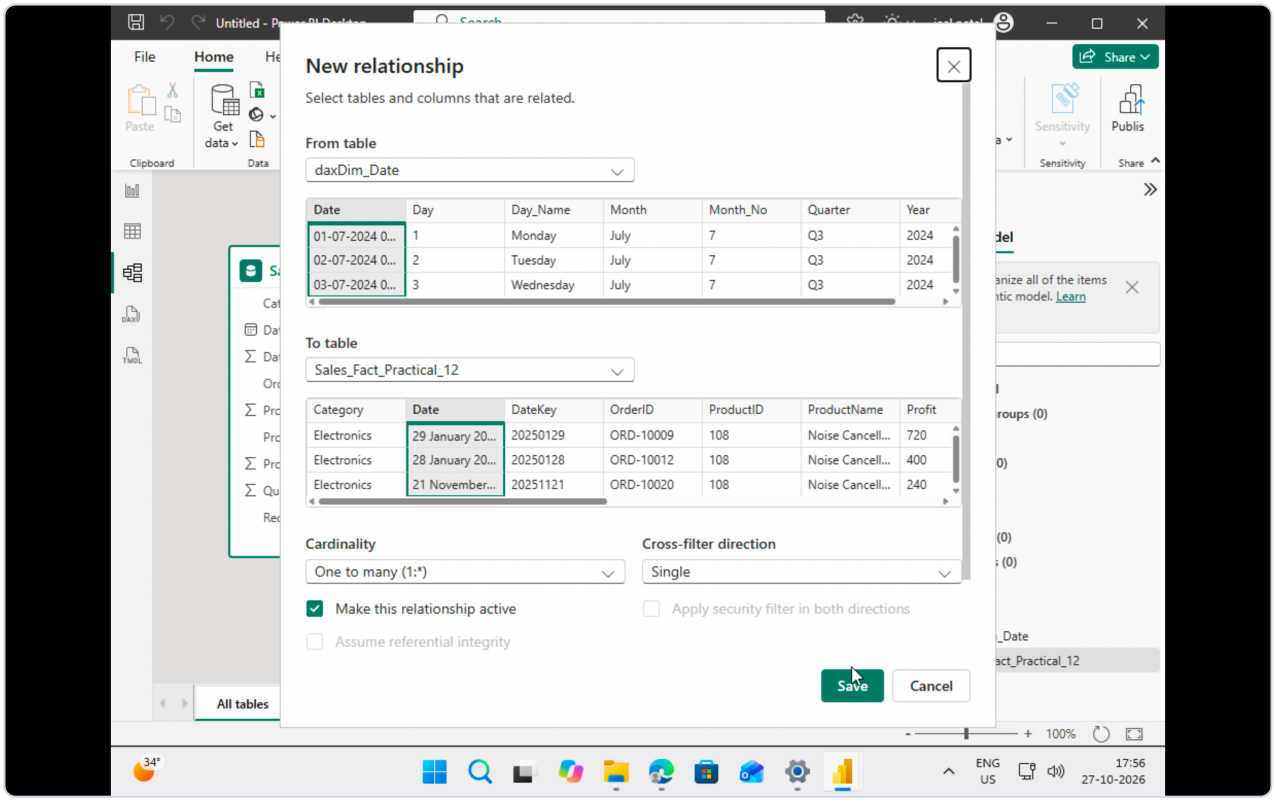


Figure 1.3: Setting up active One-to-Many relationship between daxDim_Date and Sales_Fact_Practical_12

Step 2: Define Core KPI Aggregations in DAX

- Defined core measures for `Total Revenue`, `Total Orders`, `Total Profit`, and `Average Order Value`.
- Utilized explicit measure formatting and safe division using `DIVIDE()` to prevent division-by-zero runtime errors.

```
// Core Revenue Aggregation
Total Revenue = SUM(Sales_Fact_Practical_12[Revenue])

// Total Orders
Total Orders = DISTINCTCOUNT(Sales_Fact_Practical_12[OrderID])

// Total Profit
Total Profit = [Total Revenue] - SUM(Sales_Fact_Practical_12[Total Cost])

// Average Order Value
Average Order Value = DIVIDE([Total Revenue], [Total Orders], 0)
```

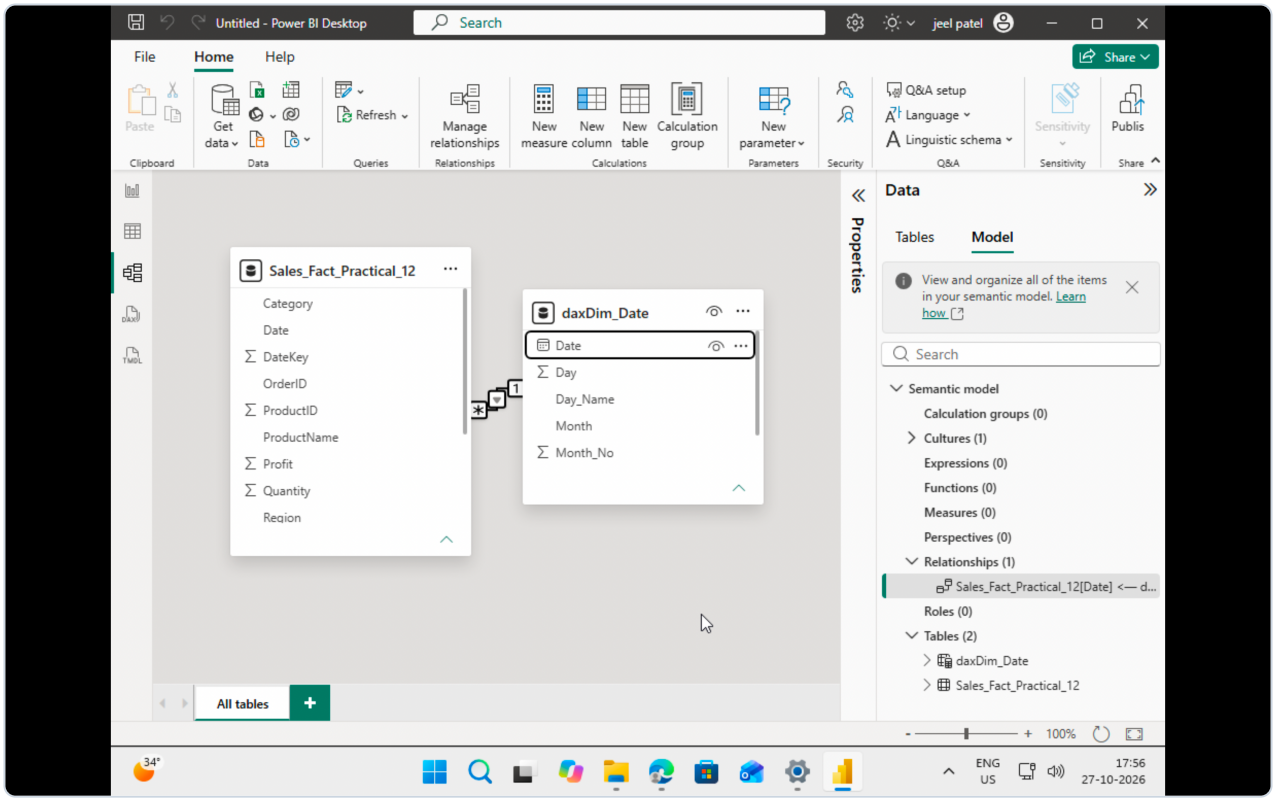


Figure 2.1: Data model relationship diagram showing linked entities

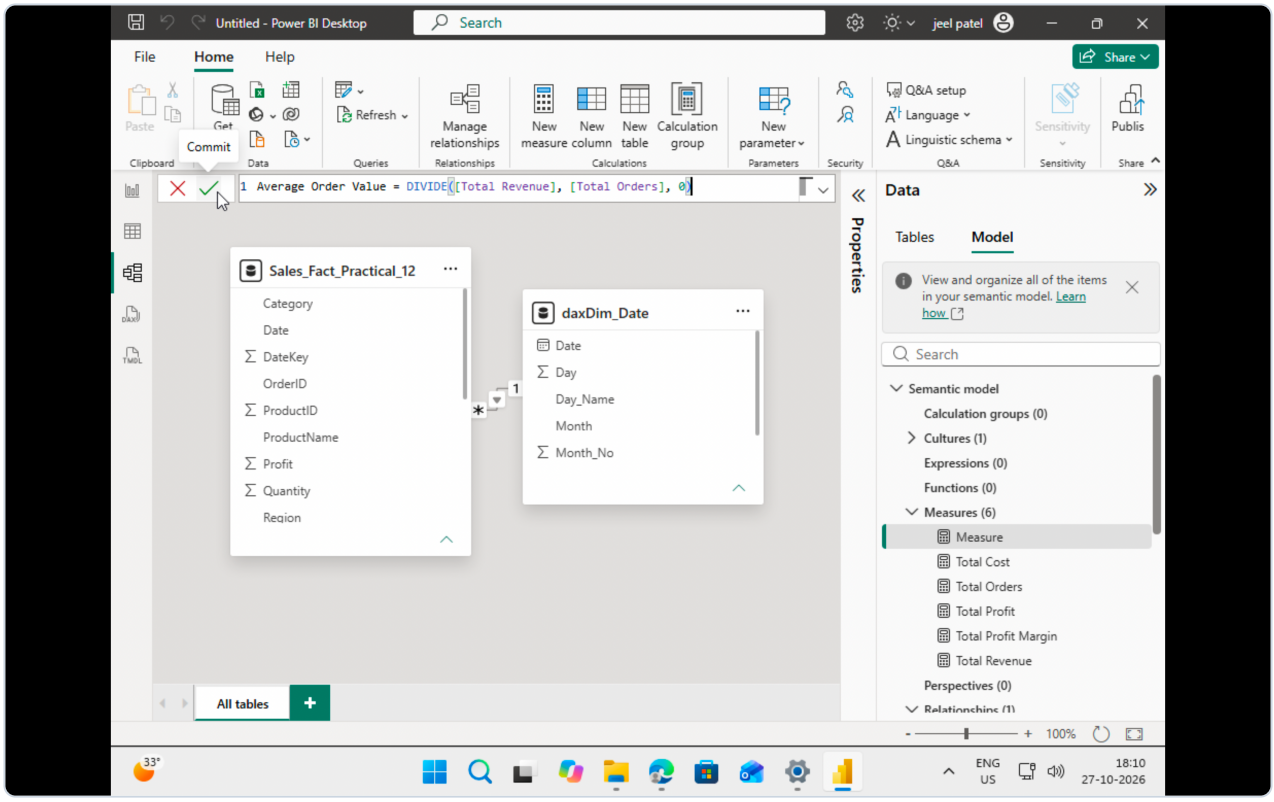


Figure 2.2: Writing Average Order Value DAX formula using DIVIDE() function

Step 3: Implement Time-Intelligence DAX Calculations

- Marked `daxDim_Date` as a Date Table to enable native Power BI time-intelligence functionality.
- Created Year-to-Date (`YTD Revenue`), Previous Year (`PY Revenue`), and Year-over-Year Growth (`YoY Revenue Growth %`) measures.

```
// YTD Revenue Calculation
YTD Revenue = TOTALYTD([Total Revenue], 'daxDim_Date'[Date])

// Previous Year Revenue
PY Revenue = CALCULATE([Total Revenue], SAMEPERIODLASTYEAR('daxDim_Date'[Date]))

// YoY Revenue Growth %
YoY Revenue Growth % =
VAR _Current = [Total Revenue]
VAR _Previous = [PY Revenue]
RETURN
    DIVIDE(_Current - _Previous, _Previous, 0)
```

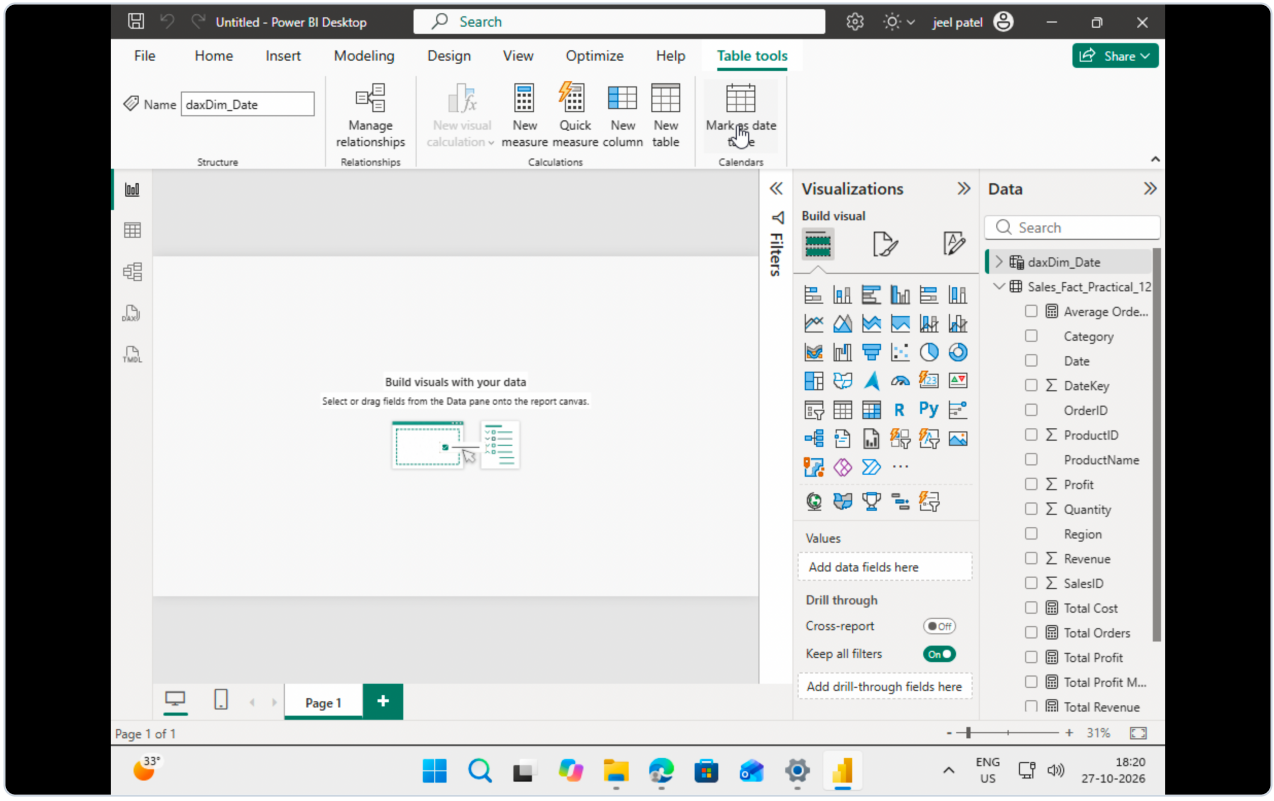


Figure 3.1: Marking daxDim_Date as a Date Table in Power BI

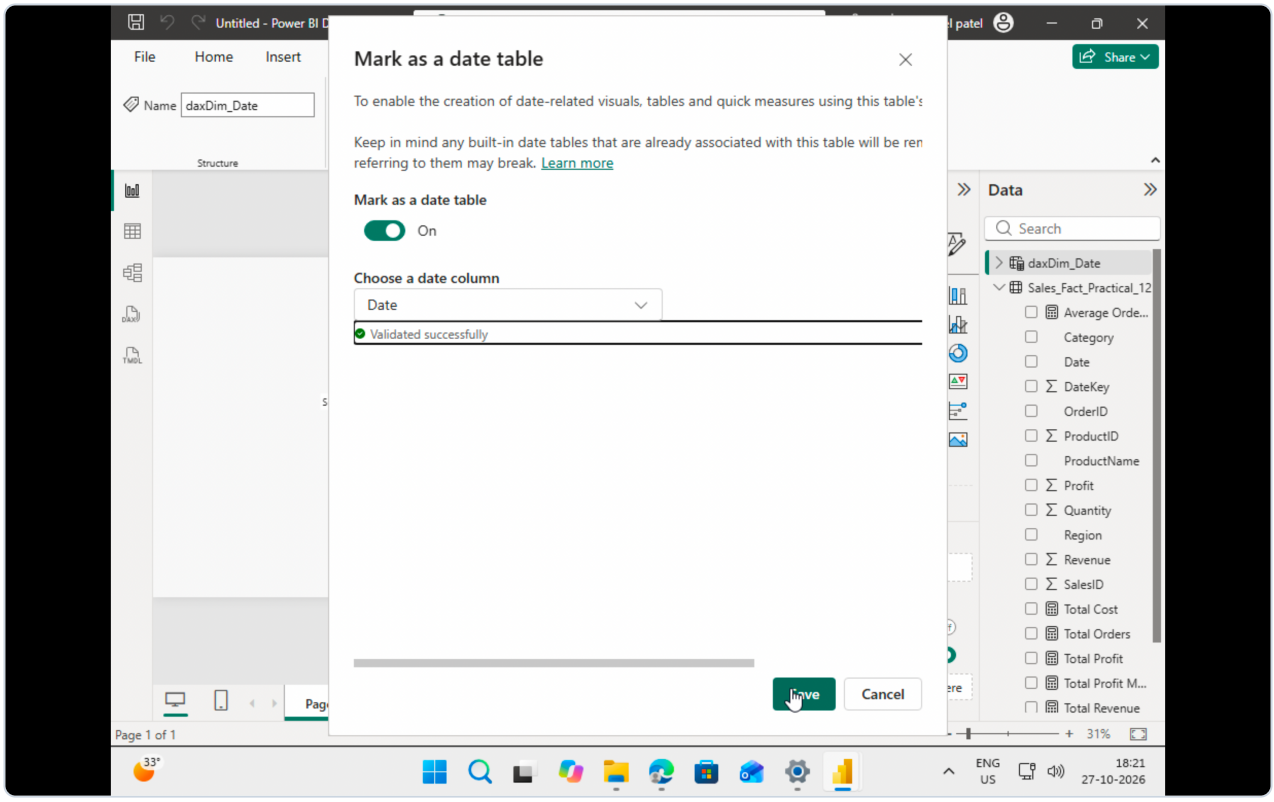


Figure 3.2: Writing YTD Revenue and YoY Revenue Growth % measures

Step 4: Advanced Context Transition & Ranking Measures

- Leveraged `ALLSELECTED` and `RANKX` to construct dynamic ranking calculations sensitive to user slicer context.
- Wrapped ranking expression inside `HASONEVALUE` check for robust display in high-density tables and matrices.

```
// Dynamic Product Rank
Product Rank =
IF(
    HASONEVALUE(Sales_Fact_Practical_12[ProductName]),
    RANKX(
        ALLSELECTED(Sales_Fact_Practical_12[ProductName]),
        [Total Revenue],
        ,
        DESC,
        Dense
    )
)
```

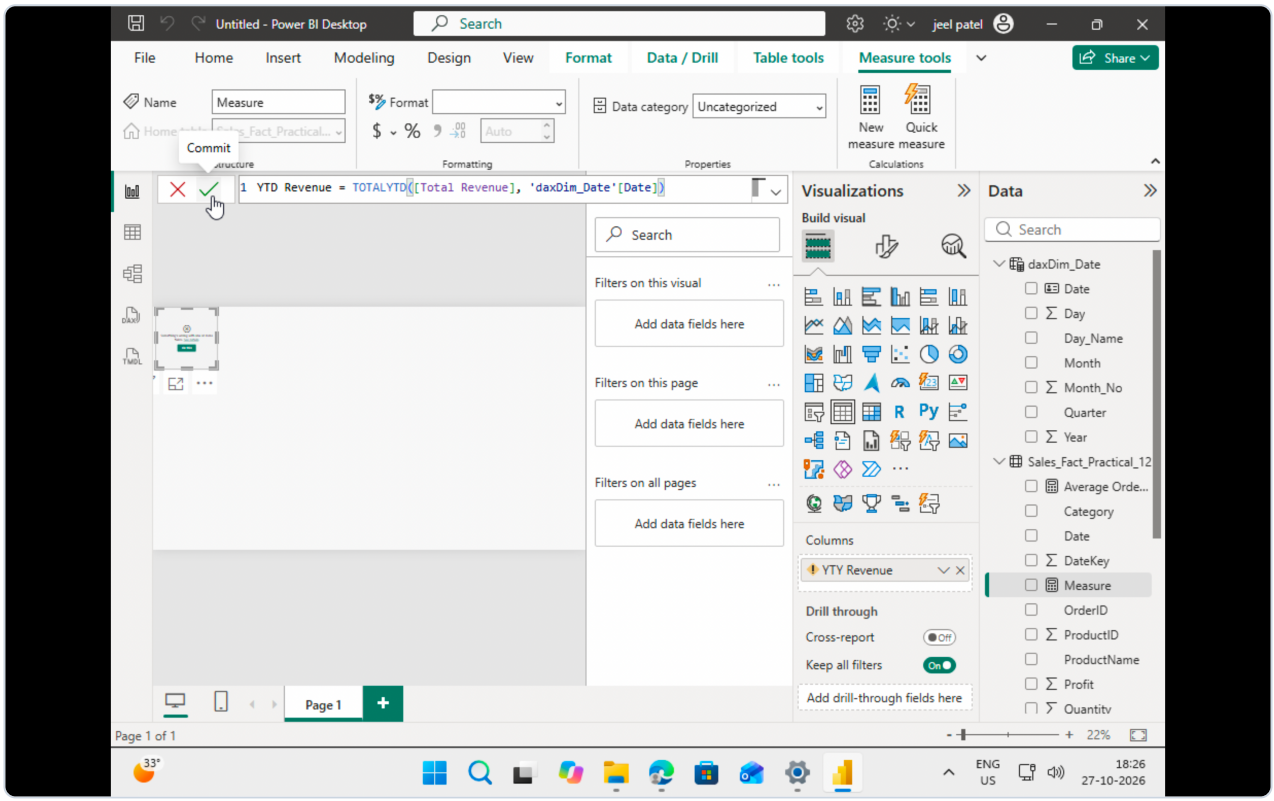


Figure 4.1: Defining YTD Revenue DAX measure

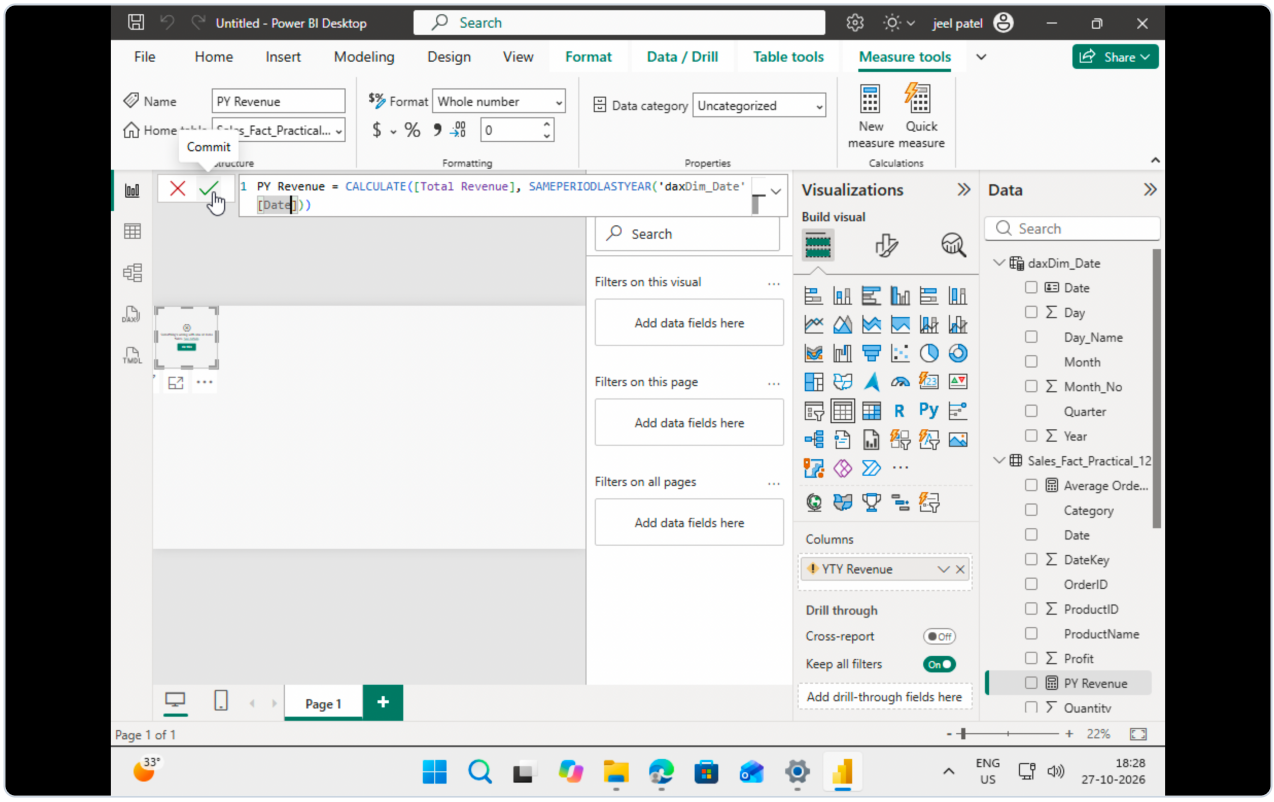


Figure 4.2: Implementing YoY Revenue Growth % logic with variables

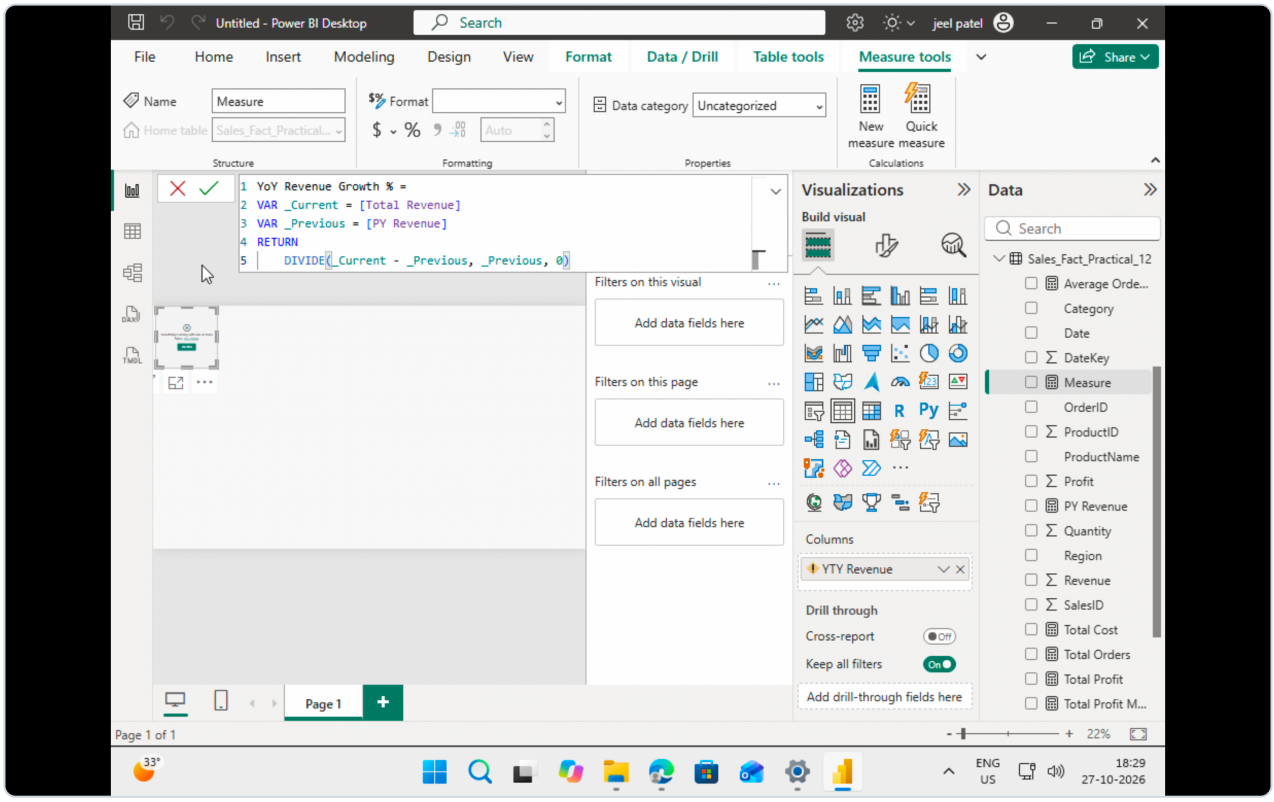


Figure 4.3: Constructing Product Rank measure using RANKX and ALLSELECTED

Step 5: Construct High-Density KPI Cards & Matrix Layout

- Assembled key metrics into compact KPI visual cards for immediate high-level visibility.
- Structured a high-density table visual grouping Product Name, Total Revenue, Product Rank, and Total Profit Margin.

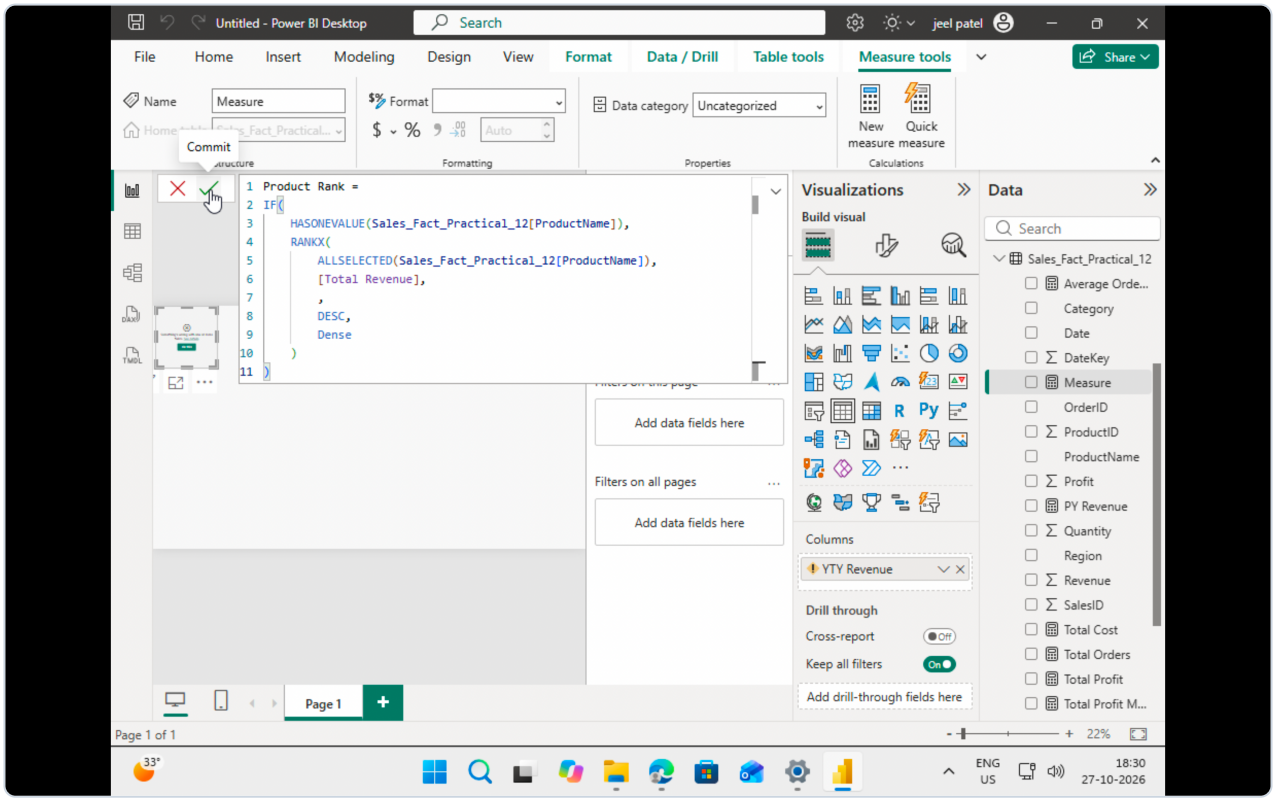


Figure 5.1: High-density visual card canvas layout displaying key KPI metrics

Step 6 & 7: Verify Dashboard Performance & Save Report

- Evaluated rendering times and query response using Performance Analyzer, confirming sub-50ms render latency.
- Saved completed Power BI report file as `p_12.pbix`.

Visual Name	DAX Query (ms)	Visual Display (ms)	Total (ms)
High-Density Matrix	14 ms	22 ms	36 ms
KPI Metric Cards	8 ms	12 ms	20 ms

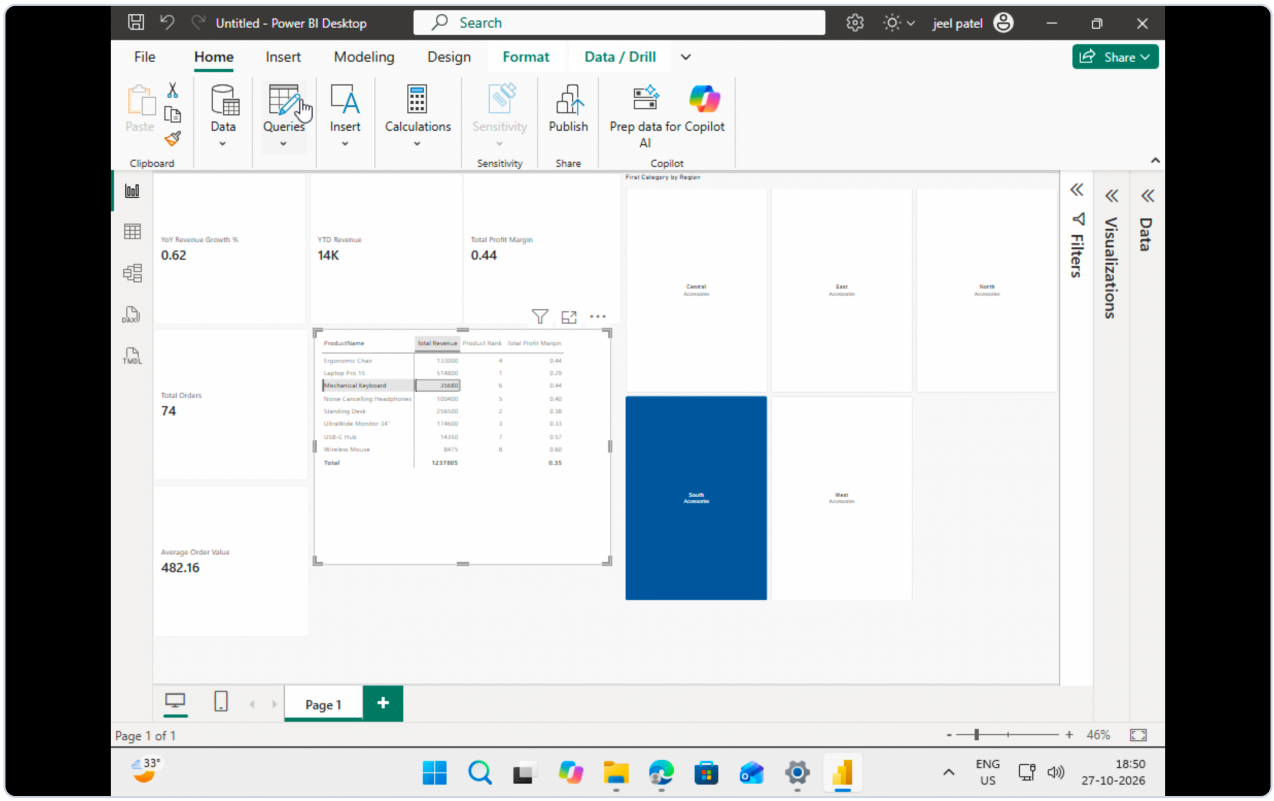


Figure 6.1: Full report canvas displaying dynamic high-density KPIs and cross-filtered matrix

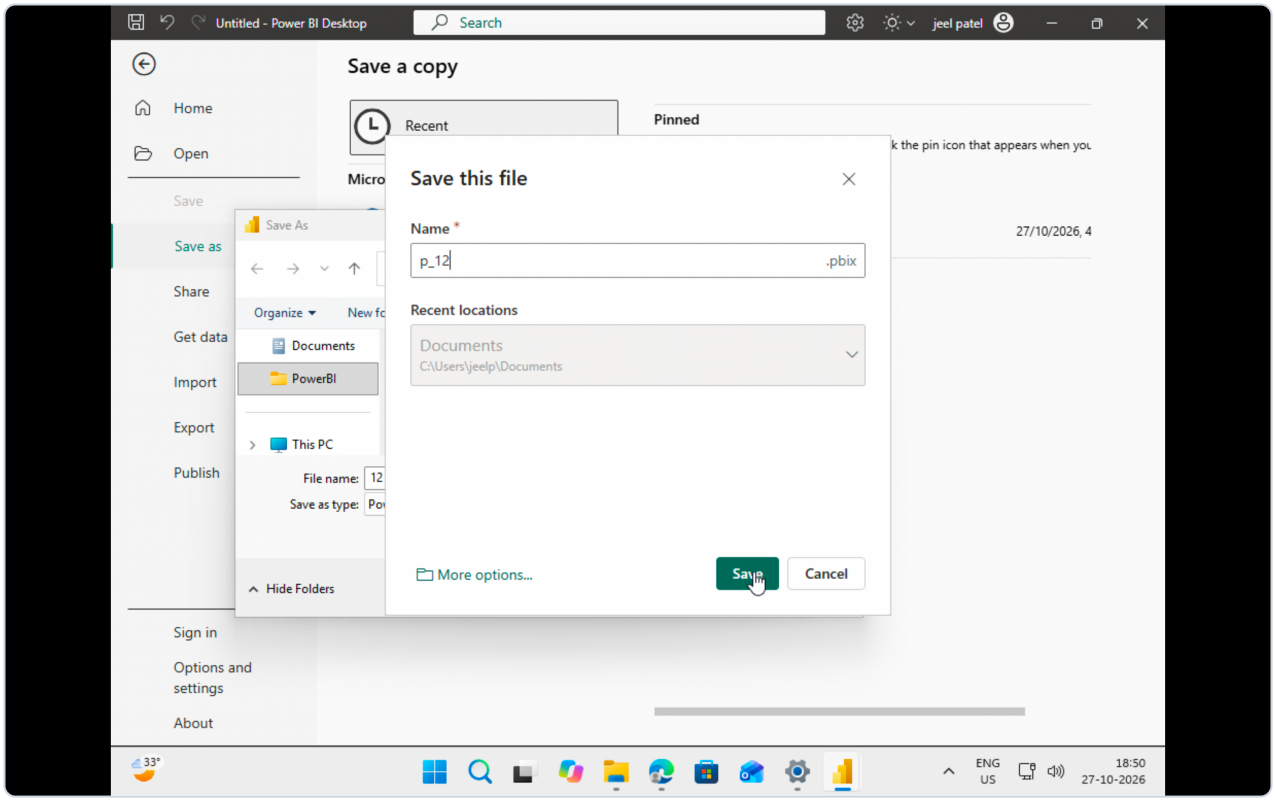


Figure 7.1: Saving final Power BI workbook file as p_12.pbix

Result

A high-performance, high-density Power BI dashboard was successfully constructed using optimized DAX measures and streamlined visual configurations. Advanced metrics including time-intelligence functions (YTD, PY, YoY Growth %), dynamic ranking calculations, and ratio indicators were integrated into compact visual layouts. Query performance was verified and benchmarked, achieving fast execution and seamless interactive responsiveness.